

UNIVERSITÄT
LEIPZIG

Principles & Methods

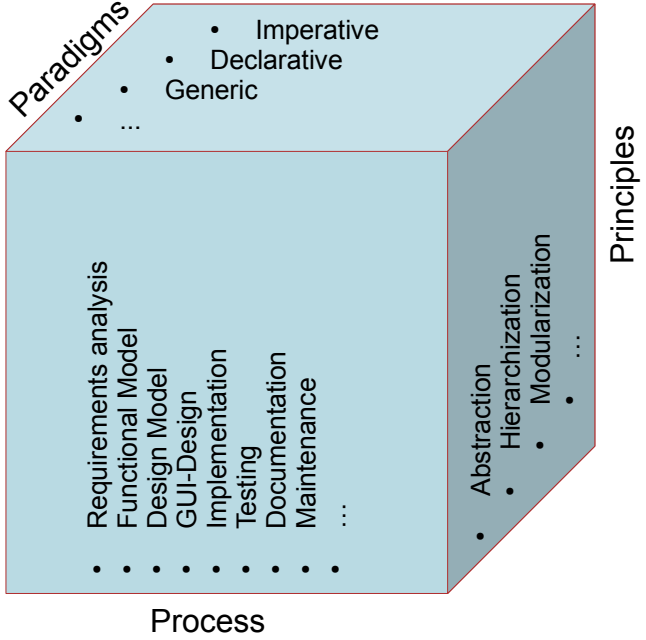
Software Engineering

Leipzig, WS 26/27
Dr. Nico Willert Stark / Prof. Dr. Ulrich Eisenecker

LICENSE

- This presentation is subject to the copyright terms of Moodle
- You agreed to these terms when registering to Moodle

DIMENSIONS OF SOFTWARE ENGINEERING



ESSENCE OF ENGINEERING

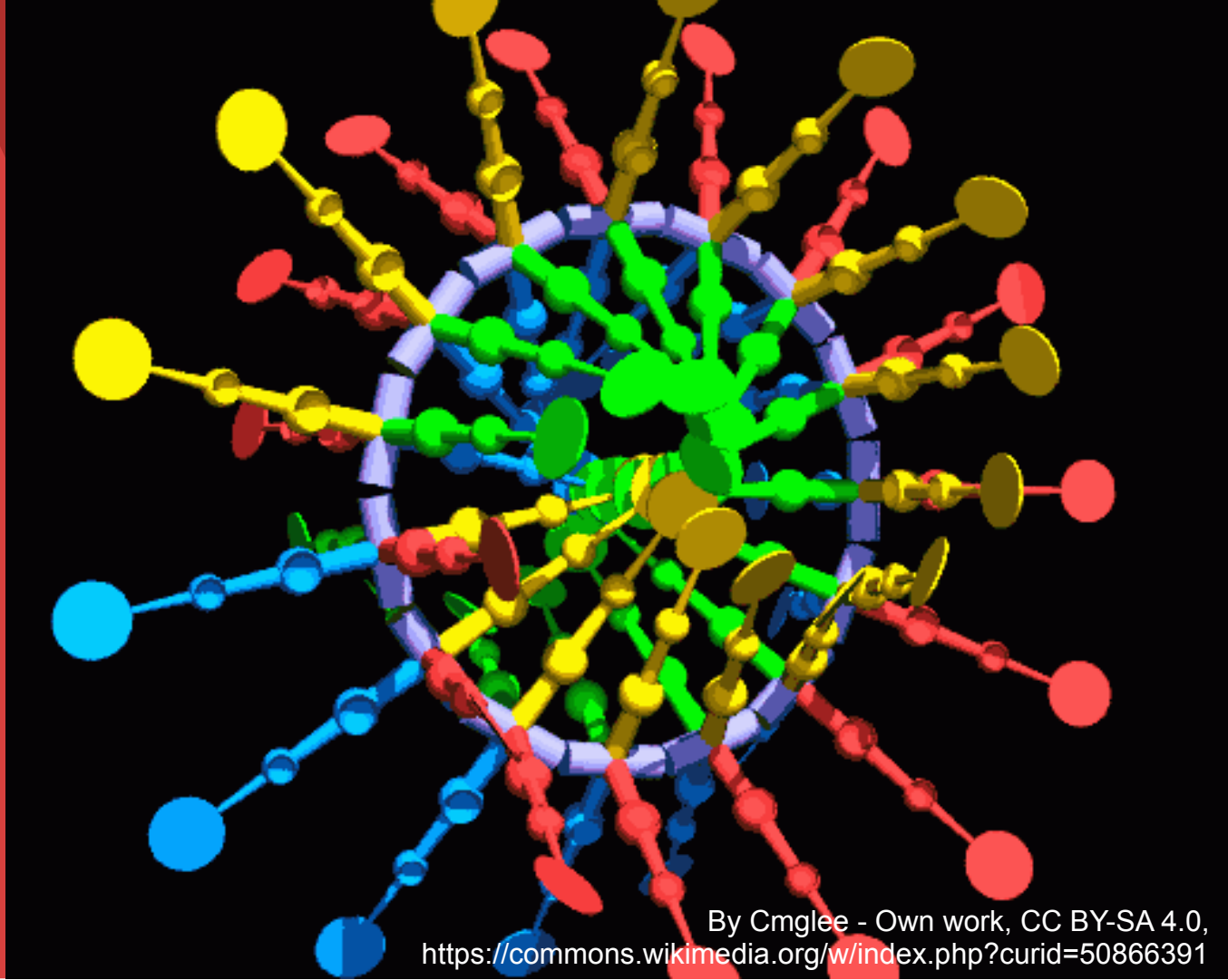
PRINCIPLES

- Are general and abstract guidelines for acting
- They build on experience and insight and may be confirmed or refuted by them
- Maybe, a principle in engineering is the analogy of an axiom in mathematics

METHODS

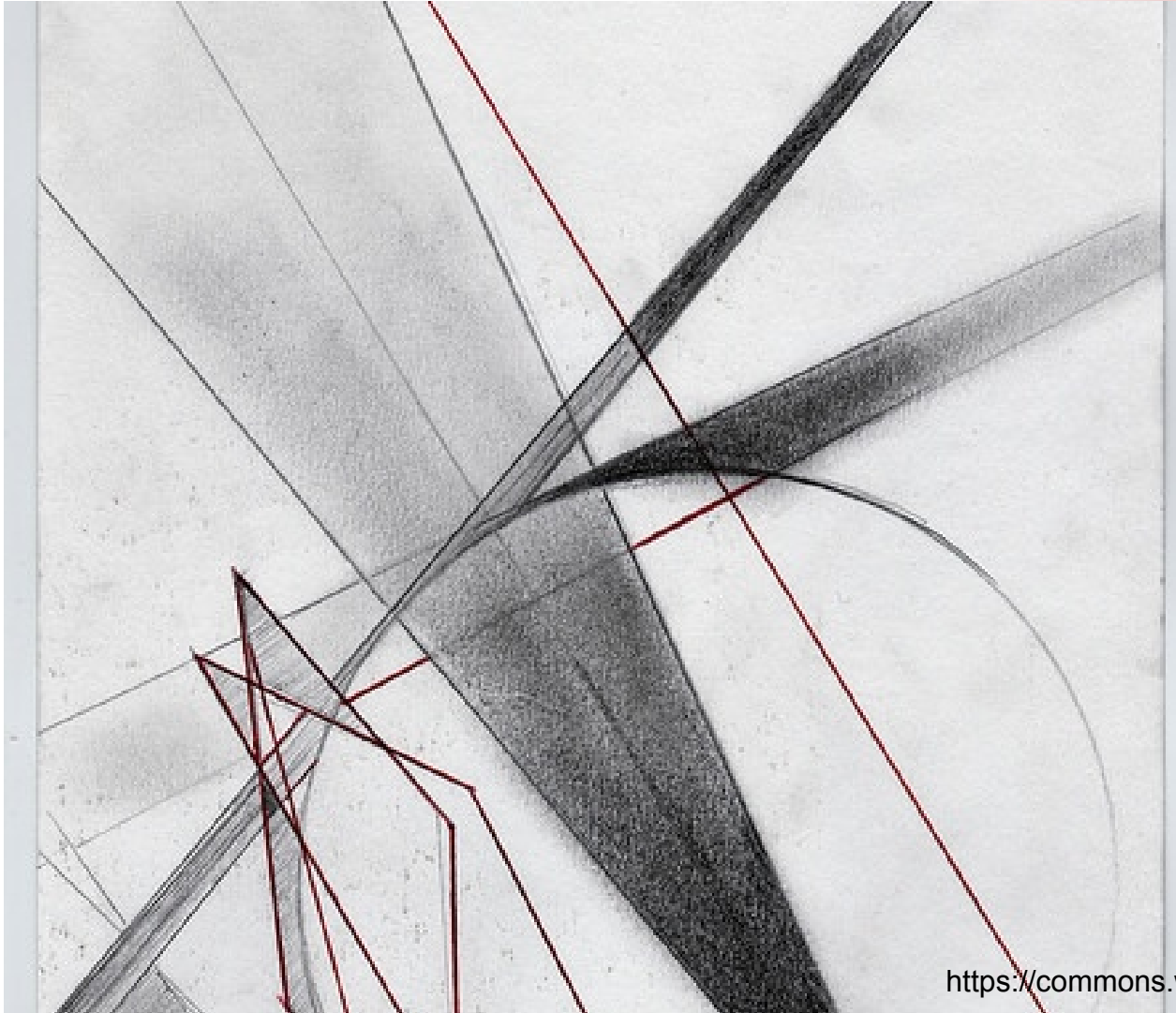
- Are consciously applied systematic approaches to accomplish a given goal
- In software engineering a method may be used as general term for
  - Concepts
  - Notations
  - Systematic approaches

PRINCIPLES



By Cmglee - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=50866391

ABSTRACTION



By V1ru2 - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=47005817

ABSTRACTION

- Abstraction focuses on essentials and omits unnecessary details
- Abstraction always assumes a certain perspective and serves a specific purpose
  - Hence, there is no universal abstraction
  - But, it is possible to have different, maybe complementary, abstractions of one and the same part of reality
- Abstraction is always relative; there may be more or less abstract levels
  - Less abstract usually means more concrete
  - Thus, abstractions form a stack of layers or levels
- The challenge in software engineering is to find the appropriate level of abstraction for a specific purpose

EXAMPLES

- Abstraction of physical bodies may assume various perspectives
  - Mass, energy, form, color, velocity, composition, …
- Abstraction levels are also differing
  - Color: spectral color, i.e., wavelength of light, color code, e.g., RGB, CMYK, HSV, qualitative, e.g., warm vs. cold
  - Form: smallest enclosing cuboid, smallest enclosing circle, approximation by grid model, qualitative, e.g., thin vs. thick
  - …
- Type and level of abstraction depend on the given problem
  - Sorting ripe and unripe cherries
  - Separating wheat from chaff
  - Using body mass index to assess the risk of heart attack



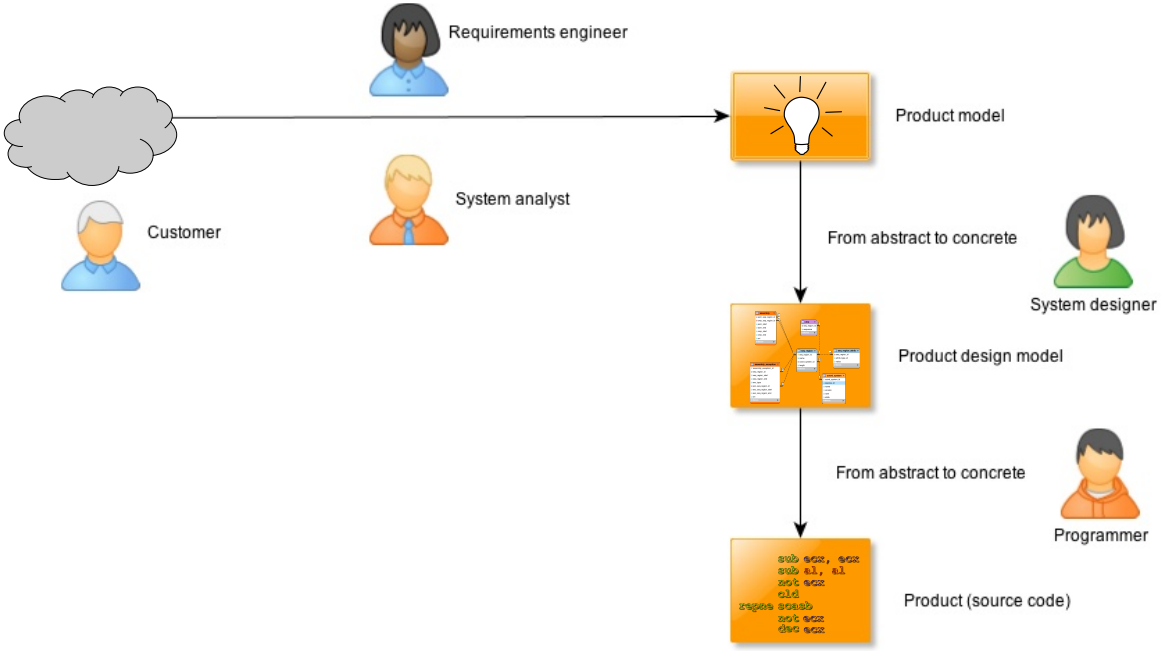




https://openclipart.org/detail/143617/cherry, modified
https://openclipart.org/detail/175150/wheat-trigo
https://openclipart.org/detail/154747/heart-attack

ABSTRACTION AND CONCRETION

- Software development requires to change permanently between abstraction and concretion

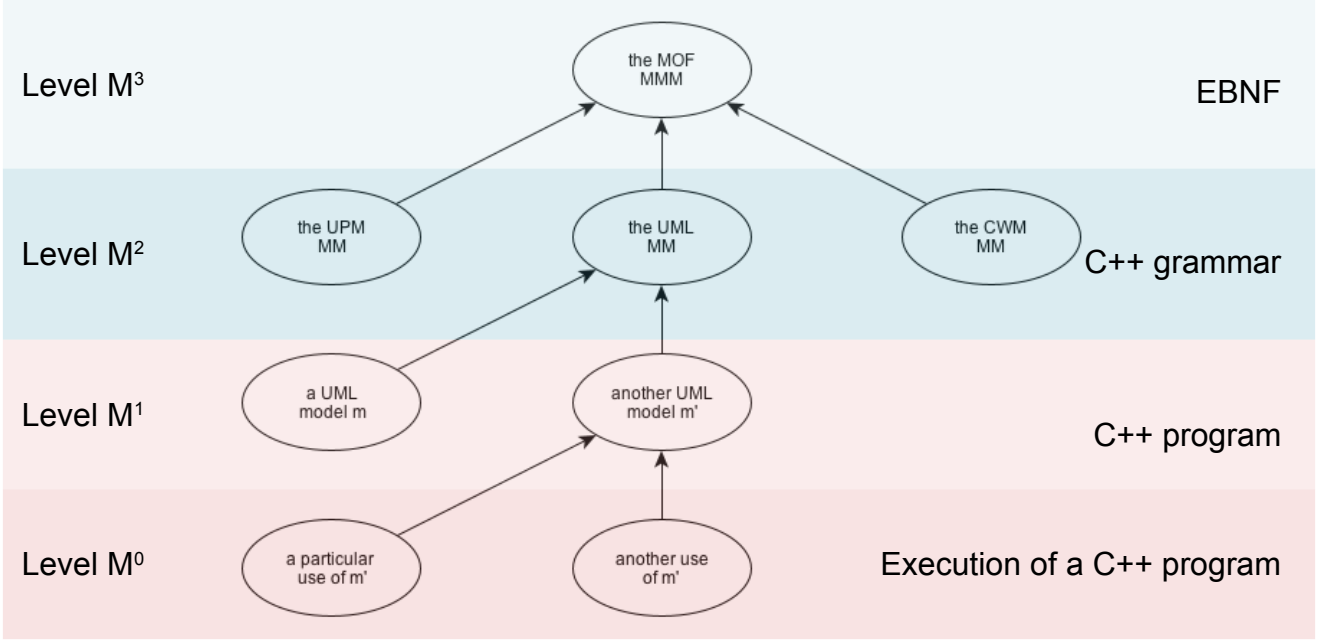


Balzert 1998, p. 562, https://openclipart.org/detail/236010/grey-cloud-1, https://openclipart.org/detail/125719/idee-idea,https://openclipart.org/detail/245897/assembly, https://openclipart.org/detail/256888/string-length

ABSTRACTION LEVELS

- Hesse et al. 1994 (according to Balzert 1998, pp. 565) distinguish three levels of abstraction in software development
  - Exemplar level
    - Concrete items, persons, or concepts, and their relations, concrete actions and events, e.g., Hans Maier owns a VW Golf
  - Type level
    - Types or classes of items, persons, or concepts, classes of relation types and events, e.g. owner keeps automobile
  - Meta-level
    - Defines concepts and their relations, which are used on the type level, e.g. class, attribute, method, association

OMG FOUR LAYERS STANDARD MODELING STACK

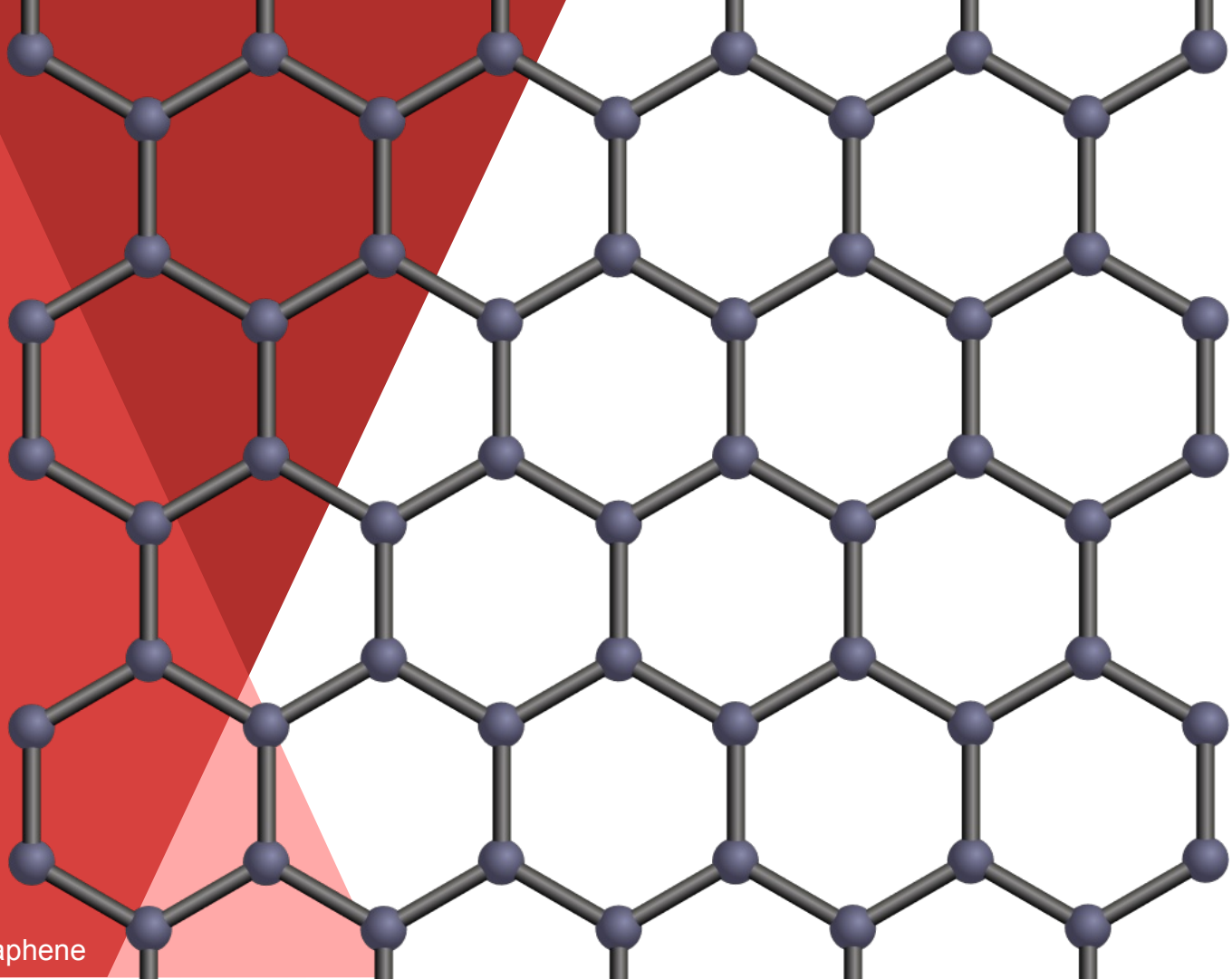


According to “About the Meta Object Facility Specification Version 1.4.” and Bézivin 2001 (modified)

ABSTRACTION VS. COARSENING

- According to Wendt (1993) it is necessary to use both means in analyzing and designing large systems
  - A coarsened, i.e., oversimplified, view of a system
    - Supports an intuitive understanding without all necessary details
    - May lead to statements that do not apply to the concrete system
    - Requires detailed definition when dealing with the concrete system
    - Example: an OOA-model may have two unrelated classes; in the OOD-model a relation between the two classes is added
- An abstract view of a system requires to add details when dealing with the concrete system
  - Example: an OOA-model has a class with attribute “date of birth”; the OOD-model adds a derived attribute named “age”

STRUCTURE

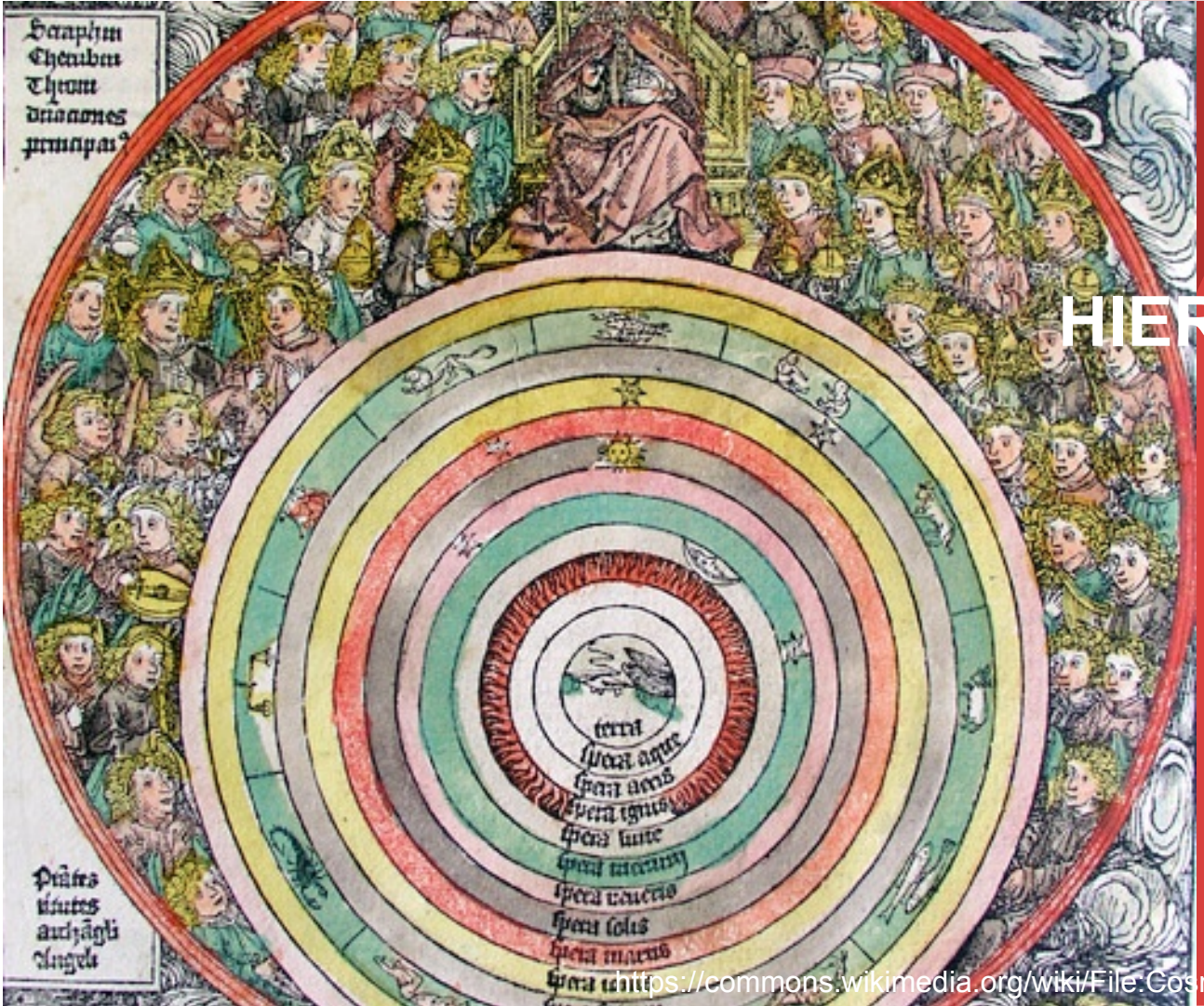


https://openclipart.org/detail/29143/graphene

STRUCTURING

- A structure consists of a system's parts and their relations
  - It can be defined by mathematical relations
  - Some graphical representations are undirected, e.g., entity-relationship diagram, some are directed, e.g., dataflow diagram, flowchart
- Three types of structures can be distinguished in software engineering
  - Static structure: documents the structure of a system and is stable from a certain point of time
  - Dynamic structure: develops during the execution of a system at runtime
  - Organizational structure: emerges with progressive development or maintenance of a system
    - It is not fixed in the beginning and cannot be planned completely

HIERARCHIZATION

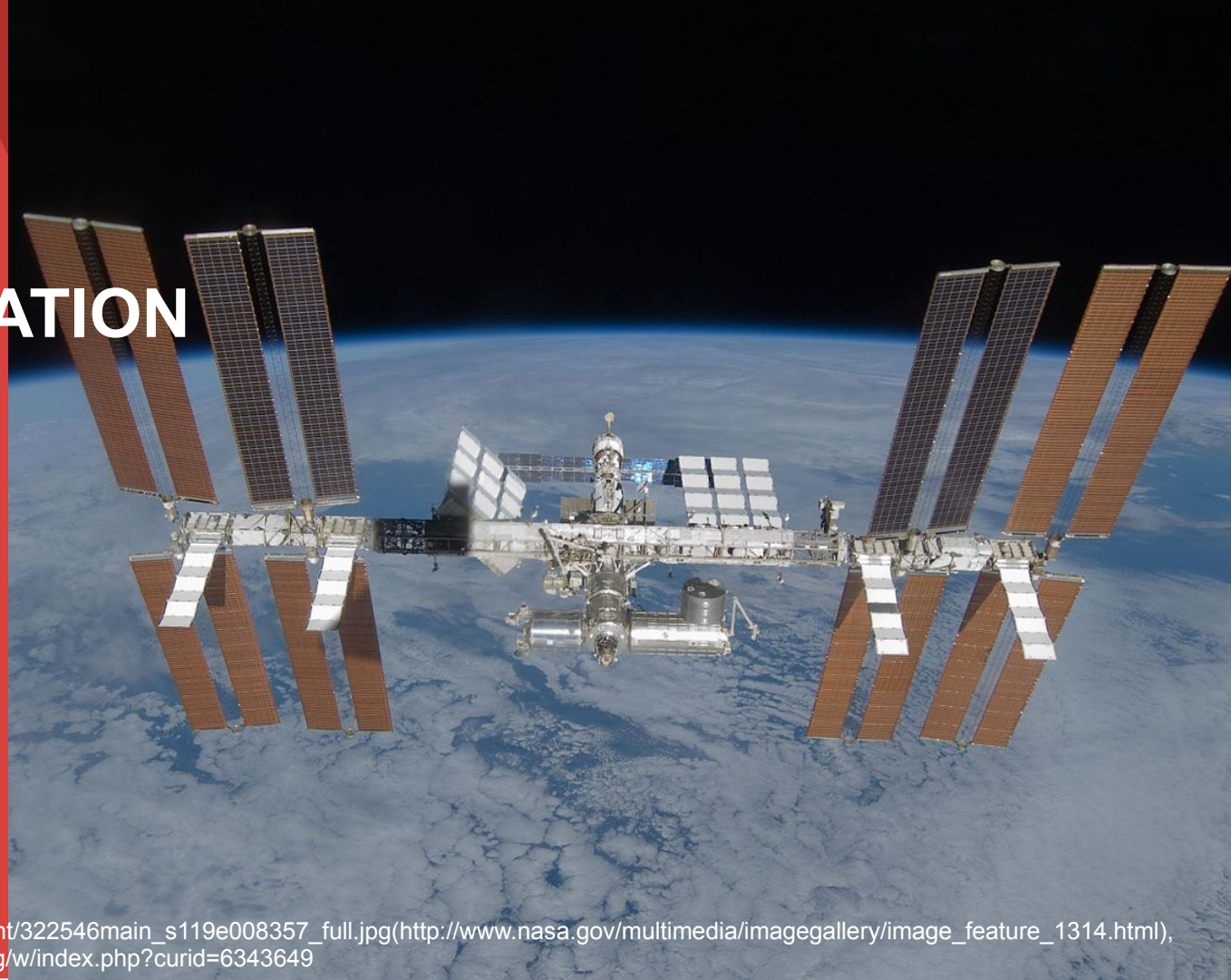


https://commons.wikimedia.org/wiki/File:Cosmos_from_The_*Nuremberg_Chronicle.jpg

HIERARCHIZATION

- Hierarchization means to rank order, superordinate, or subordinate
  - It refines the principle of structuring
  - Relations among parts of a hierarchy are always directed
- Parts of the same rank form a level or layer of the hierarchy
- Like in structuring there are three types, namely static, dynamic, and organizational hierarchy
- Graphical representations of hierarchies are usually organized from top to bottom, less frequently from left to right, e.g., inheritance hierarchies, aggregation trees, decision trees
- A certain element can be part of several hierarchies simultaneously, e.g., class B inherits from A and a B contains an A

MODULARIZATION



By NASA - http://www.nasa.gov/images/content/322546main_s119e008357_full.jpg(http://www.nasa.gov/multimedia/imagegallery/image_feature_1314.html), Public Domain, https://commons.wikimedia.org/w/index.php?curid=6343649

MODULARIZATION (1)

- Elements of a system are organized in so-called modules
- Each module defines an interface, which is the only way for using it
- Cohesion is a measure that describes how closely the elements of a module are related to each other
  - A module with high cohesion is a functional unit with semantically associated elements
- Coupling is a measure for dependencies among the interfaces of modules
  - A low coupling means that a module is context independent, i.e., it can be developed, understood, tested, and serviced independently of other modules
- A general design goal is to maximize cohesion and to keep coupling as low as possible

MODULARIZATION (2)

- In case of error correction, maintenance, or enhancements the entire module is replaced, removed, or added, but not single elements of it
  - Hence the interface of the module must be well-defined and may not be altered
- Modularization can be done on small and large scales (Balzert 1998, p. 573)
  - Identification and creation of modules requires abstraction
  - Modularization implies the formation of levels of abstraction
  - The result is a hierarchical and recursive decomposition into modules (“divide and conquer”)
- Modules foster alignment of project plans and teams
  - After modules are conceived and their interfaces are defined, it is possible to identify project activities that can be parallelized and those, that have to be carried out sequentially

POSSIBLE PROBLEMS OF MODULARIZATION

- Available means for modularization sometimes do not suffice to structure a system optimally from each perspective
  - E.g., a functional class hierarchy of container classes should be extended with optional tracing and optional logging
    - Using classes only results in many new classes combining functional and non-functional features, that is, tracing and logging can not be encapsulated as classes
    - Aspect-oriented programming overcomes this deficit by encapsulating tracing and logging in aspects
- Modularization is probably the most important software engineering principle
  - It is related to the principle ”separation of concerns”



INFORMATION HIDING

By Roger prat - Own work, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=18441436

INFORMATION HIDING

- Information hiding (Parnas 1971) refines and tightens modularization
- An abstract data type (ADT) encapsulates operations and attributes
  - An exemplar of an ATD can be accessed only by its public operations, which – altogether – constitute its public interface
    - This contributes to context independency of an ADT
  - Attributes and private operations including their implementations are not accessible or completely hidden from the user
    - The liberal variant allows users to see private members of an ADT, but prohibits their use; this may lead to undesired dependencies
    - The strict variant conceals all private members of an ADT from the user; this requires to separate definition and implementation of an ADT

CONSEQUENCES

- Accessing protected base class members in a derived class also breaks information hiding and yields possibly fragile designs; thus
  - Inheritance considered harmful!
- Friend-declarations (as in C++) also violate information hiding and should be avoided
- Consequent application of information hiding keeps coupling between modules low, thus supporting a robust design
- Sometimes it is difficult to fully obey information hiding, e.g.,
  - Unit tests of private or protected methods
  - Declarative access of private or protected attributes, such as in object query language expressions

LOCALITY



By Antonio Pérez de Aguilar – PainterDetails of artist on Google Art Project - 1AEcwQ1XSN1qLw at Google Cultural Institute maximum zoom level, Public Domain, https://commons.wikimedia.org/w/index.php?curid=21850938

LOCALITY

- A human being, and a software engineer as well, can only deal with a limited amount of information simultaneously
  - Magical Number Seven, Plus or Minus Two (Miller 1956)
  - Hence, all information required for understanding a problem should be located
    - In a single place and
    - Ideally fit to one paragraph or page
- Good locality means that relevant information is available at a single point and must not be “constructed”, nor by browsing, neither by searching, or jumping
  - Furthermore, there should be no irrelevant information, hiding or cluttering relevant information, e.g., code tangling

IMPROVING LOCALITY AND CONFLICTS

- Defining a variable close to its use improves locality, e.g.,
  for (int i = 0; i <= 10; ++i) …
- But, there are also conflicts
  - Complete definitions of public operations (prototype plus implementation) can reduce locality when only the prototypes are relevant for understanding the public interface of a class, e.g., for assessing coupling
  - Separating the definition of a class and the implementation of its operations in different files can reduce locality if it is required to understand all members of a class and their interaction, e.g., for assessing cohesion
- Tools can help to overcome this conflict, e.g., folding editors or toggling the display of attribute and operation compartments in case tools

Das Erst Buch Mose. I.

I.



IN anfang schuff Gott himel vnd erden/Vnd die erde war wüst vnd leer/vnd es war finster auff der tiefe/vnd der Geist Gottes schwebet auff dem wasser.

Vnd Gott sprach/Es werde liecht/ Vnd es ward liecht/vnd Gott sahe das liecht fur gut an/Da scheidet Gott das liecht vom finsternis/ vnd nennet das liecht/Tag/vnd die finsternis/Nacht/ Da ward aus abend vnd morgen der erste tag.

Vnd Gott sprach/Es werde eine feste zwisschen den wassern/vnd die sey ein vnterscheid zwisschen den wassern/Da macht Gott die Feste/vnd scheidet das wasser hunden/von dem wasser droben an der Festen/Vnd es geschach also/Vnd Gott nennet die Festen/Himel/ Da ward aus abend vnd morgen der ander tag.

Vnd Gott sprach/Es samle sich das wasser vnter dem himel/an sondere örter/das man das trocken sehe/ vnd es geschach also/ Vnd Gott nennet das trocken/Erde/vnd die samlung der wasser nenner er/ Meere/Vnd Gott sahe es fur gut an.

Vnd Gott sprach/Es lasse die erde auff gehen gras vnd kraut/das sich besame / vnd fruchtbare beume/ da ein jglicher nach seiner art frucht trage/vnd habe seinen eigen samen bey jm selbs/auff erden/ Vnd es geschach also/Vnd die erde lies auff gehen/gras vnd kraut/ das sich besamet/ein jglichs nach seiner art/vnd beume die da frucht trugen/vnd jren eigen samen bey sich selbs hatten/ein jglicher nach seiner art/Vnd Gott sahe es fur gut an/Da ward aus abend vnd morgen der dritte tag.

Vnd Gott sprach/Es werden Liechter an der Feste des Himels/ vnd scheiden tag vnd nacht / vnd geben/zeichen/monden/tage vnd jare/vnd seien liecher an der Festen des himels/das sie scheinen auff er-

Zeichen / als der sonnen/vnd monden finsternis /

By Published by Hans Luft, 1534 - Luther Bible of 1534, Public Domain, https://commons.wikimedia.org/w/index.php?curid=60489058

VERBALIZATION

VERBALIZATION

- Appropriate names, e.g., for identifiers, are essential for understanding source code, models, architectures, and documentation
  - Once a name is chosen it will persist for a long time
  - A good name is meaningful, concise, and intentional
  - Names are for humans, not for tools
  - Conventions and rules for formation and spelling should be respected, e.g., camel case notation
    - If tools force naming conventions for this reason, it is ok
- Verbalization also applies to comments, self documenting concepts, methods, and languages
- An obfuscator is a tool for alienating names by replacing them with automatically generated names, which are unique within their context, but meaningless for humans

SEPARATION OF CONCERNS



By Jon Sullivan, PD Photo; This color separation by Jacob Rus - Original photograph from PD Photo, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=4688862

SEPARATION OF CONCERNS

- Problems and their solution usually have multiple dimensions
  - As pointed out in the context of locality, humans can only deal with a limited amount of information at the same time
  - This also applies to problem-solving, when understanding a problem and devising a solution it is advisable to cope only with one dimension at a time
- Dijkstra called this principle “separation of concerns” (Dijkstra 1976), e.g.,
  - Correctness of a program, identification and elimination of errors, and run-time efficiency, etc., should be each addressed in isolation
  - Such an isolated part should be examined and modelled in its entirety in a problem adequate way
- Separation of concerns is the core principle of aspect-oriented programming

PROS AND CONS OF SEPARATION OF CONCERNS

✅ Like modularization, separation of concerns helps to manage complexity
- Planning and organization of work as well as responsibilities can be aligned according to concerns

⚠ Sometimes it is difficult to separate concerns in a way, that the resulting modularization is problem adequate
- Identification of concerns
- Separating primary from secondary concerns
- If appropriate tools, e.g., aspect weavers, are not available, it can be difficult or impossible to implement programming concerns in a pristine way

⚠ Possibilities for global optimizations can be overseen when dealing with isolated concerns only

$$I = \int_{a}^{b} f(x)dx = \int_{a}^{x_4} f(x)dx = \int_{a}^{a+4h} f(x)dx, \text{gdzie} x_i = a + i\text{horazh} = \frac{(b-a)}{4}$$

$$I = \int_{a}^{a+4h} f(x)dx = h[4f(a) + 8\Delta f(a) + (\frac{64}{3} - 8)\frac{\Delta^2 f(x_0)}{2!} + (64 - 64 + 16)\frac{\Delta^3 f(x_0}{3!}$$
$$+ \frac{704}{3} - 48)\frac{\Delta^4 f(x_0)}{4!}]$$

$$I = [4f(a) + 8(f(a+h) - f(a)) + \frac{20}{3}(f(a+2h) - 2f(a+h) + f(a)) + \frac{8}{3}(f(a+3$$
$$+ 3f(a+h) - f(a)) + \frac{14}{45}(f(a+4h) - 4f(a+3h) + 6f(a+2h) -$$

$$I = \frac{h}{45}[14f(a) + 64f(a+h) + 24f(a+2h) + 64f(a+3h) + 14f(a+4h)]$$

$$I = \frac{2h}{45}[7f(a) + 32f(a+h) + 12f(a+2h) + 32f(a+3h) + 7f(a+4h)]$$

$$I = \frac{2h}{45}(7f(a) + 32f(x_1) + 12f(x_2) + 32f(x_3) + 7f(b))$$

RIGOR AND FORMALITY

By Azathi - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=37403553

RIGOR AND FORMALITY

- According to Ghezzi et al. (2003, pp. 42) this principle helps to transform software development into a well-defined, reliable, traceable, and repeatable process
  - Rigor means precision and accuracy and complements creativity
  - Formalization means to provide formal descriptions of products and processes in a mathematical representations
    - Mathematical descriptions can be validated and proofed more easily, e.g., termination of a loop
- Despite the advantages of this principle its exaggerated application has serious drawbacks
  - It is not effective to mathematically formalize instable or volatile requirements
  - Often, a pervasive high level of formalization if is not required
  - Mathematical models may be difficult to understand for some stakeholders

ANTICIPATION OF CHANGE

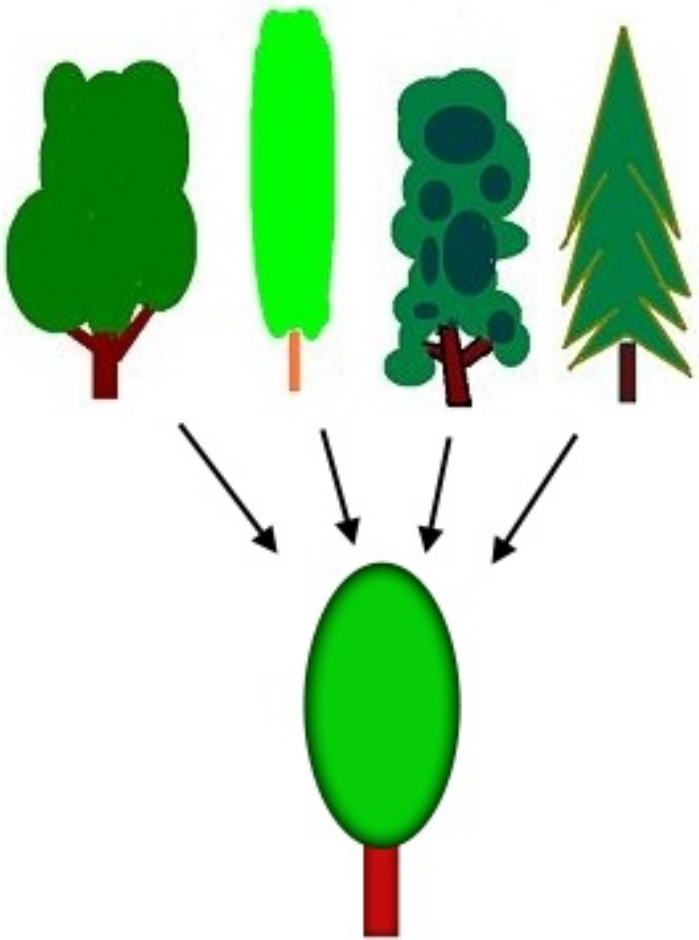


By Metropolitan Transportation Authority of the State of New York - Blizzard of 2015: Bus Snow Prep, CC BY 2.0, https://commons.wikimedia.org/w/index.php?curid=39510961

ANTICIPATION OF CHANGE

- In order to adapt to a changing environment and new requirements software and its development processes may frequently undergo changes
- Anticipation of change (Ghezzi et al. 2003, pp. 50) emphasizes a design of processes and products that embrace expected or highly probable changes
  - Avoid unnecessary dependencies
  - Non-reversible decisions should not be done prematurely
  - Parts of systems or processes with a high probability of change should be isolated from stable parts
- Examples
  - Personal turnover can be addressed by by work contracts, a sound and incremental documentation, and a project plan considering training for new staff
  - Replacement of system software can be addressed by checking plans, e.g., for changing a DBMS, and thus provide modular and flexible access to the possibly changing part

GENERALITY



By Tomwsulcer - Own work, CC0, https://commons.wikimedia.org/w/index.php?curid=14710171

GENERALITY

- Generality builds upon abstraction and guides the search for and the development of solutions (Ghezzi et al. 2003, p. 52)
  - First, generalize the given problem, i.e., abstract from its irrelevant details
  - Second, search a solution for the generalized problem
    - If a solution is found, use it and adapt it as necessary
    - If no solution is found, develop a general solution to the generalized problem yourself, provided, the investment is economically feasible
- Good examples for applying generality are catalogues of design patterns; a single catalogue entry comprises
  - The actual pattern, i.e., the description of a generic problem-solution pair, and
  - Concrete, specialized examples of its application

PROS AND CONS OF GENERALITY

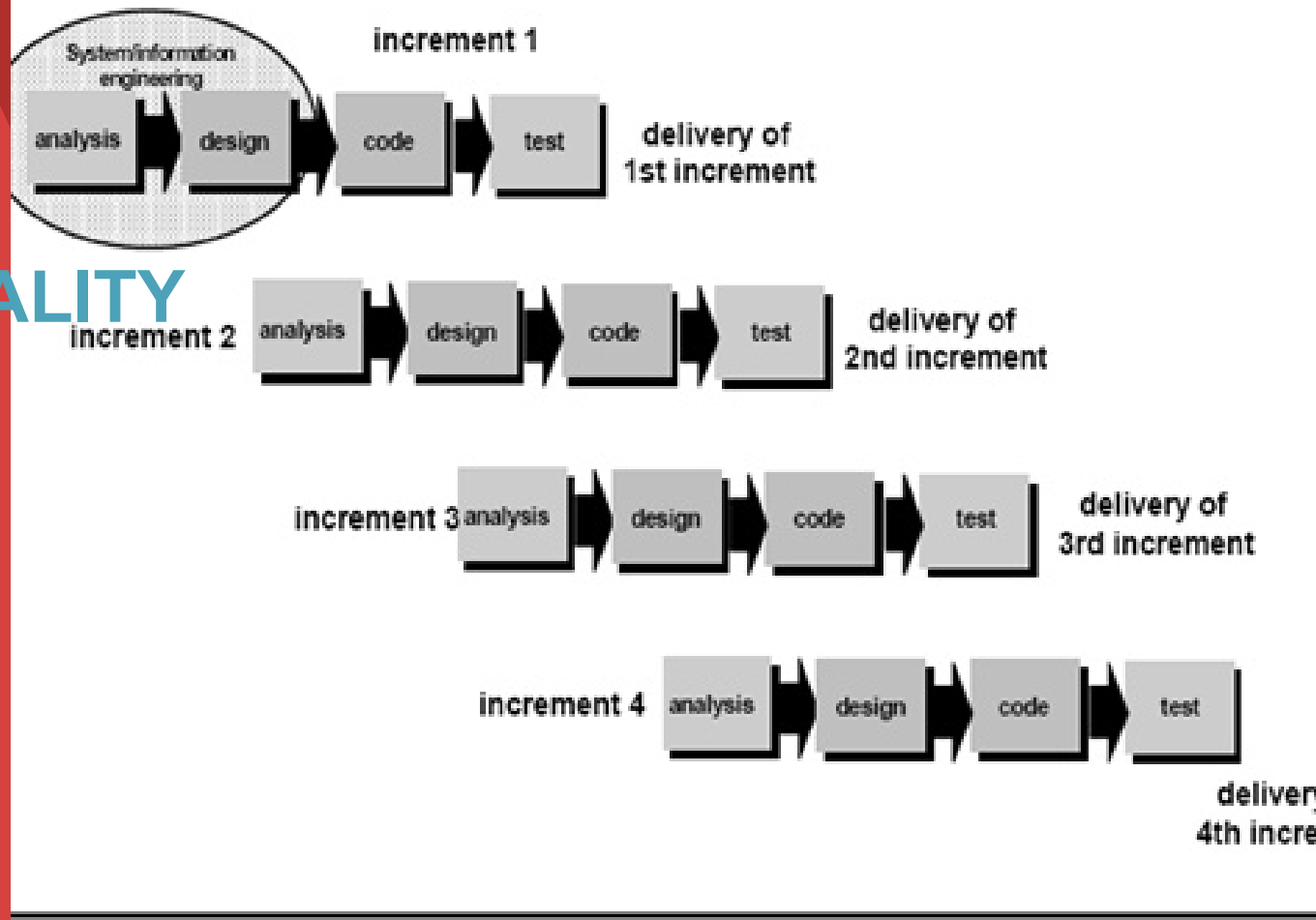
PRO

- Generality is a remedy against re-inventing the wheel
- Reusing proven solutions can reduce cost and shorten time to market
- Prevents development of many specialized, yet similar solutions, thus reducing maintenance effort

CONTRA

- Striving for generality can be time consuming and expensive
- Benefits are lost when over-generalizing
- Generalized solutions can be less efficient and less performant than specialized solutions

INCREMENTALITY



By macromediax.com - http://www.macromediax.com/learn/archive.asp?id=191, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=17829026

INCREMENTALITY

- Incrementality means “piecemeal development” (Ghezzi et al. 2003, pp. 53)
  - Developing product increments allows users to give an early feedback for the actual increment (provided instant delivery to and its test by the customer)
  - This supports early identification and improvement of inconsistent, incomplete, erroneous, or misunderstood requirements
    - The later in the software life cycle an error is detected, the higher are the incurred costs
  - New increments can be built easily on already existing, stable increments
    - This reduces the risk of integration problems, provided, increments are continuously integrated and tested
  - Usually, all development activities have to be executed when developing a new increment

DON’T REPEAT YOURSELF



By Think Draw from Dubai, UAE - Flickr.com - image description page, CC BY 2.0, https://commons.wikimedia.org/w/index.php?curid=346312

DON'T REPEAT YOURSELF (DRY)

- DRY demands that "Every piece of knowledge must have a single, unambiguous, authoritative representation within a system." (Hunt & Thomas 2000, p. 27)
  - Duplicates in software artifacts, which have been manually introduced without intention, have serious drawbacks, e.g., clones in source code
  - They increase the size of the artifact and reduce its comprehensibility and maintainability
  - If a change is required, e.g., due to error correction, all duplicates have to be updated manually and consistently, otherwise the quality of the artifact degrades
- If strict modularization is required and dependencies among modules have to be avoided (in order to achieve low coupling), redundancy in order to preserve independent development may be acceptable
- Important: automatically introduced redundancy, e.g., by generating source code, is not subject to DRY

AND MORE PRINCIPLES …



By NASA, ESA, E. Jullo (JPL/LAM), P. Natarajan (Yale) and J-P. Kneib (LAM). - http://www.spacetelescope.org/images/heic1014a/, CC BY 3.0, https://commons.wikimedia.org/w/index.php?curid=11561821

IMPLEMENTATION PRINCIPLES (BALZERT 1998, PP. 928)

- Verbalization (see previous slides)
- Problem-specific datatypes, i.e., sufficient elementary datatypes, user-defined datatypes, such as enumeration types, structures (a.k.a. records), unions (a.k.a. variant record), classes, arrays
  - Optimal is a 1:1 mapping of datatypes and –structures from problem space to implementation, e.g., enum days { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY };
- Stepwise refinement (Wirth 1971, Dijkstra in Dahl et al. 1972, pp. 1 – 82)
- Integrated documentation and its peak “literate programming” by Donald Knuth
  - http://www.literateprogramming.com offers a wealth of information, examples, and tools including documentation

MY FAVORITE THREE PRINCIPLES BEYOND ABSTRACTION AND MODULARIZATION

- Responsibility
  - Responsibility is a relation between you, possibly as a representative, and another entity, e.g., a person or a software artifact
  - If it exists and as long as it exists you are in charge of this entity with respect to important properties, e.g., its well-being, maintainability, and so on
- Automation
  - If there is potential for automation with respect to some activity or artifact, realize it – provided this is technically possible and economically feasible
- Redundancy (strongly related or equal to DRY)
  - If there is verbatim or structural redundancy in manually created artifacts or activities carried out by humans eliminate it – provided this is technically possible and economically feasible
  - Don’t care about redundancy which is automatically created

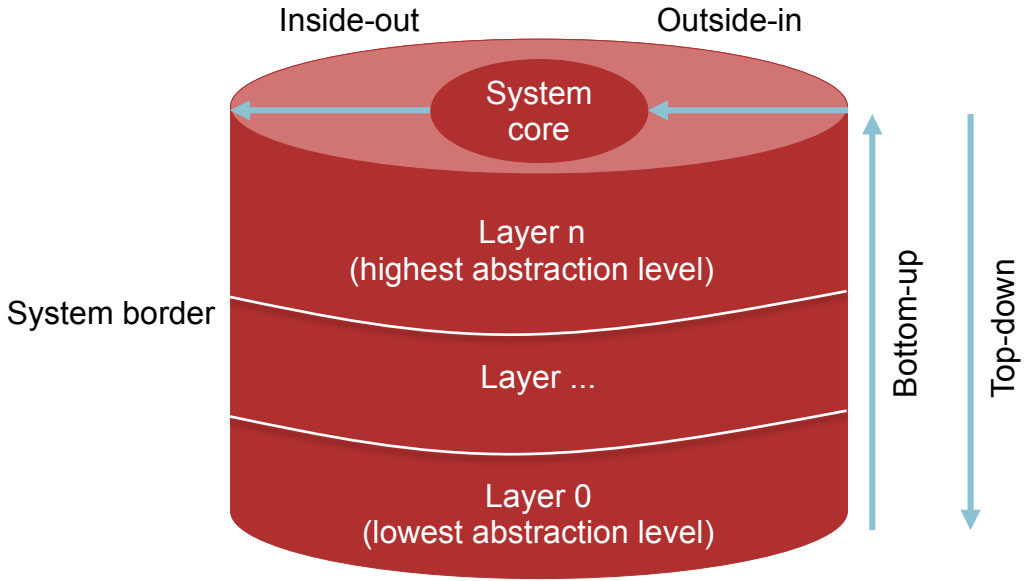
METHODS



https://openclipart.org/detail/191659/gps-satellites-trilateration

METHODS

- There are two dimensions for classifying methods, (well-defined procedures to accomplish a goal)
  - Direction of abstraction, i.e., bottom-up vs. top-down
  - System perspective, i.e., outside-in vs. inside-out



TOP-DOWN METHOD

- Appropriate for creating software from scratch, i.e., no preceding system is available and there are new and emergent requirements
- Pros
  - Helps to focus on essential aspects, details are neglected in the first step
  - It is made easier to recognize structure
- Cons
  - Requires high abstraction capability
  - Decisions may be delayed or delegated to inappropriate, less abstract levels
  - Sometimes it is difficult to identify the top abstraction level reliably

BOTTOM-UP METHOD

- Is appropriate if a similar or a legacy system already exists
- Pros
  - A concrete starting basis is available
  - A certain part of requirements is known, understood, and already implemented
  - Reuse may be facilitated
- Cons
  - Superordinate structures may be hidden by details
  - In order to achieve the goal, the given basis has to be broad enough

OUTSIDE-IN METHOD

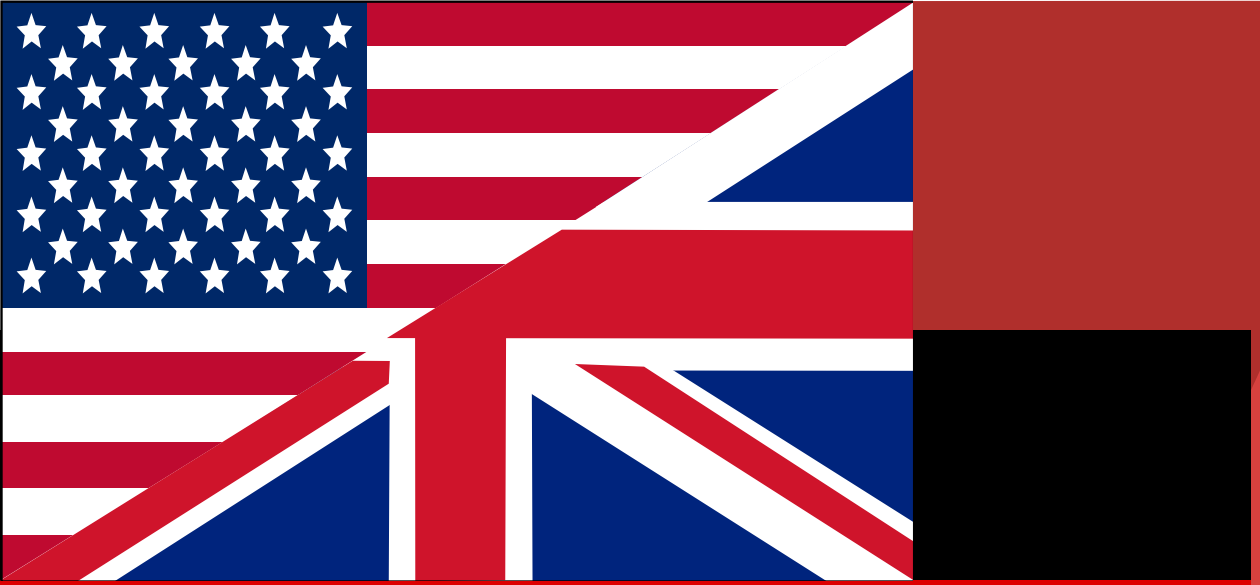
- Similar to top-down this method is appropriate for developing entirely new systems
- Pros
  - Helps to focus on interfaces of the system and the services it offers for the actors
  - Relations between system and environment are not overlooked
- Cons
  - Environment may distract from internal problems
  - Reusability may be restrained

INSIDE-OUT METHOD

- Similar to bottom-up this method is appropriate if there is a similar or a preceding system
- Pros
  - Focus is on structure
  - It is possible to design and implement a stable system core early, which supports adding further system parts around this core incrementally
- Cons
  - Threat to overlook requirements coming from environment or actors or to recognize them too late

MIXING METHODS

- Which method is best depends on the development task
  - After setting up a first top level structure of the system under development it may be useful to select different methods for different sub-systems
  - Furthermore, it is possible to combine methods, e.g., bottom-up and inside-out for developing a system when similar or preceding systems are available
- The chosen method should support making decisions at the right time, especially not prematurely
- The chosen method and the development progress achieved with its application should be thoroughly documented for revision



GLOSSARY

https://openclipart.org/detail/168121/usuk-flag
https://openclipart.org/detail/296539/german-flag

- Complementary: (sich) ergänzend
- Cuboid: Quader
- Wheat: Weizen
- Chaff: Spreu
- Coarsened: vergröbert
- To emerge: sich herausbilden, hervor kommen
- To alter: verändern
- Alignment: Ausrichtung
- To conceive: erdenken, entwerfen
- To suffice: genügen, ausrichten
- To tighten: verschärfen
- To conceal: verbergen
- To yield: ergeben, erbringen
- Concise: Präzise, prägnant
- Intentional: absichtlich, vorsätzlich
- Obfuscator: Verdunkler, Verfinsterer

- To devise: entwickeln
- Pristine: makellos, rein, unverfälscht
- Rigor: Genauigkeit, Präzision
- Pervasive: durchdringend, allgegenwärtig
- Stakeholder: Interessenvertreter
- To embrace: umarmen, annehmen
- Personal turnover: Mitarbeiterfluktuation
- Remedy: Abhilfe, Gegenmittel

REFERENCES



By Stewart Butterfield - flickr, CC BY 2.0, https://commons.wikimedia.org/w/index.php?curid=1093147

REFERENCES (1)

- “About the Meta Object Facility Specification Version 1.4.” Accessed September 17, 2018. https://www.omg.org/spec/MOF/1.4/.
- Balzert, Helmut, and Helmut Balzert. Software-Management, Software-Qualitätssicherung, Unternehmensmodellierung. Lehrbuch der Software-Technik, Helmut Balzert ; 2. Heidelberg: Spektrum Akad. Verl, 1998.
- Bézivin, Jean. From Object Composition to Model Transformation with the MDA., 2001. https://doi.org/10.1109/TOOLS.2001.10021.
- Dahl, Ole-Johan, Edsger Wybe Dijkstra, and C. A. R. Hoare. Structured Programming. A.P.I.C. Studies in Data Processing, No. 8. London, New York: Academic Press, 1972.

REFERENCES (2)

- Dijkstra, E.W. A Discipline of Programming. Prentice-Hall Series in Automatic Computation. Prentice-Hall, 1976. https://books.google.de/books?id=nZpQAAAAMAAJ.
- Ghezzi, Carlo, Mehdi Jazayeri, and Dino Mandrioli. Fundamentals of Software Engineering. 2. ed., internat. ed. Upper Saddle River, N.J.: Prentice-Hall Pearson Education International, 2003.
- Hesse, Wolfgang, Georg Barkow, Hubert von Braun, Hans-Bernd Kittlaus, and Gert Scheschonk. “Terminologie Der Softwaretechnik, Ein Begriffssystem Für Die Analyse Und Modellierung von Anwendungssystemen, Teil 1: Begriffssystematik Und Grundbegriffe.” Informatik Spektrum 17, no. 1 (1994): 39–47.

REFERENCES (3)

- Hunt, Andrew, and David Thomas. The Pragmatic Programmer: From Journeyman to Master. Reading, Mass: Addison-Wesley, 2000.
- “Inheritance Considered Harmful!” In Guide to the Unified Process Featuring UML, Java and Design Patterns, 363–81. London: Springer-Verlag, 2003. https://doi.org/10.1007/1-85233-856-3_23.
- “Literate Programming.” Accessed October 10, 2018. http://www.literateprogramming.com/.
- Miller, George A. “The Magical Number Seven, plus or Minus Two: Some Limits on Our Capacity for Processing Information.” Psychological Review 63, no. 2 (1956): 81–97. https://doi.org/10.1037/h0043158.

REFERENCES (4)

- Miller, George A. “The Magical Number Seven, plus or Minus Two: Some Limits on Our Capacity for Processing Information.” Psychological Review 63, no. 2 (1956): 81–97. https://doi.org/10.1037/h0043158.
- Parnas, David Lorge. “Information Distribution Aspects of Design Methodology.” In IFIP Congress, 1971.
- Wendt, S. “Defizite Im Software Engineering.” Informatik-Spektrum 16, no. Februar (1993): 34–38.
- Wirth, Niklaus. “Program Development by Stepwise Refinement.” Communications of the ACM 14, no. 4 (April 1, 1971): 221–27. https://doi.org/10.1145/362575.362577.

YOUR QUESTIONS, PLEASE?



UNIVERSITÄT
LEIPZIG

THANK YOU VERY MUCH!

Prof. Dr. Ulrich Eisenecker
Professur für Wirtschaftsinformatik, insbes. Softwareentwicklung
Für Wirtschaft und Verwaltung
Grimmaische Straße 12, 04109 Leipzig, Germany
T +49 341 97-33720 F +49 341 97-33729

eisenecker@wifa.uni-leipzig.de
Professur Softwareentwicklung für Wirtschaft und Verwaltung